DevSecOps pipeline: final report

Student: Oussama Koudhai, 4NIDS6 (Esprit). Application: Spring Boot student-management API on Azure.
Repository: https://github.com/oussamakou/DEVOPS-Esprit-4NIDS6
Report built 2026-11-05 from revision `a27abdc`.

Every push runs the stages below. Each tool leaves its own report (log, result record and measured values) in the run's artifact **devsecops-security-reports**, and a status email is sent at the end of every master run. Screenshots are real captures of the runs and report pages cited under each image. Where a screenshot does not exist yet, this report says so instead of showing a substitute.

Pipeline at a glance

#	Stage	Technique	Tool	Blocks the pipeline when	Report	Status
A. Commit and pre-build security (before Maven compiles anything)						
1	Threat modelling	STRIDE threat model, reviewed by a human before coding	documentation/THREAT-MODEL.md	The document must exist; CI preserves it with the run's reports	`precommit/THREAT-MODEL.md`	**Verified in a real run**
2	Pre-commit hooks	Local hooks replayed in CI: YAML/XML validity, merge markers, private-key headers, staged-secret scan	pre-commit, Gitleaks (staged snapshot)	Any hook failure stops the pipeline	`precommit/report.html`	**Verified in a real run**
3	Static application security testing	SAST with project rules for command execution, concatenated SQL and disabled TLS verification	Semgrep	Any finding fails (--error --strict)	`sast/report.html`	**Verified in a real run**
4	Secret scanning	Full Git history scan with redacted output	Gitleaks	Any new finding fails; one exact historical fingerprint is documented in SECURITY.md	`secrets/report.html`	**Verified in a real run**
5	Dependency analysis and SBOM	Software composition analysis of the source POM with a CycloneDX inventory	Trivy	HIGH/CRITICAL fails; empty or unversioned inventory is rejected	`dependencies/report.html`	**Verified in a real run**
6	Infrastructure-as-code policy	Terraform fmt/validate, policy scan, custom SSH-to-internet rule, workflow lint, Caddy config validation	Terraform, Checkov, actionlint	Any unexcepted failure; ten exclusions are documented in SECURITY.md	`iac/report.html`	**Verified in a real run**
7	Collect pre-build reports	Aggregate every scanner output into one readable index	scripts/render-security-reports.py	Runs even after a failed gate	`index.html`	**Verified in a real run**
B. Build, quality and artifact						
8	Compile, test and coverage	Unit and HTTP integration tests with coverage measurement	Maven, JUnit (Surefire), JaCoCo	Any failed test blocks the image build	`build/report.html`	**Verified in a real run**
9	Code quality gate	Static analysis with a blocking quality gate	SonarQube Community Build	The pipeline waits for the gate and fails if it fails	`quality/report.html`	**Partly verified**
10	Artifact repository	Publish the tested JAR, then compare the stored checksum with the tested JAR	Sonatype Nexus	A different stored checksum fails the stage	`artifact/report.html`	**Partly verified**
11	Image build and vulnerability gate	Build a non-root image from the tested JAR, smoke-test it on ephemeral k3s, scan the final image	Docker, k3d, Trivy	Any HIGH/CRITICAL finding fails, including unfixed ones	`image/report.html`	**Verified in a real run**
C. Delivery to Azure						

12	Image registry	Push the scanned image and deploy by immutable digest	Azure Container Registry, OIDC login	The digest must belong to the pushed image	`registry/report.html`	**Partly verified**
13	Deployment	Docker Compose started on the app VM through Azure Run Command, pulling with the VM's managed identity	Azure Run Command, Docker Compose	The guest must print DEPLOY_SUCCEEDED and a real database-backed API request must pass	`deploy/report.html`	**Verified in a real run**
D. Acceptance, monitoring and notification						
14	Network acceptance	Scoped TCP exposure test from outside and inside Azure against an allow-list	Nmap	Unexpected open, missing required or incomplete scans fail	`acceptance/report.html`	**Verified in a real run**
15	Monitoring: Prometheus and Grafana	Verify scrape targets, sample real queries after light API traffic, list provisioned dashboards	Prometheus, Node Exporter, Grafana	API target and both host exporters UP, Grafana database healthy, both dashboards provisioned	`monitoring/`, `prometheus/`, `grafana/` `report.html`	**Implemented, not yet run live**
16	Email notification	Status, per-stage results and the run link sent at the end of every master run	SMTP over STARTTLS	Runs even after failure or cancellation	`the email, and the job in the run graph`	**Partly verified**

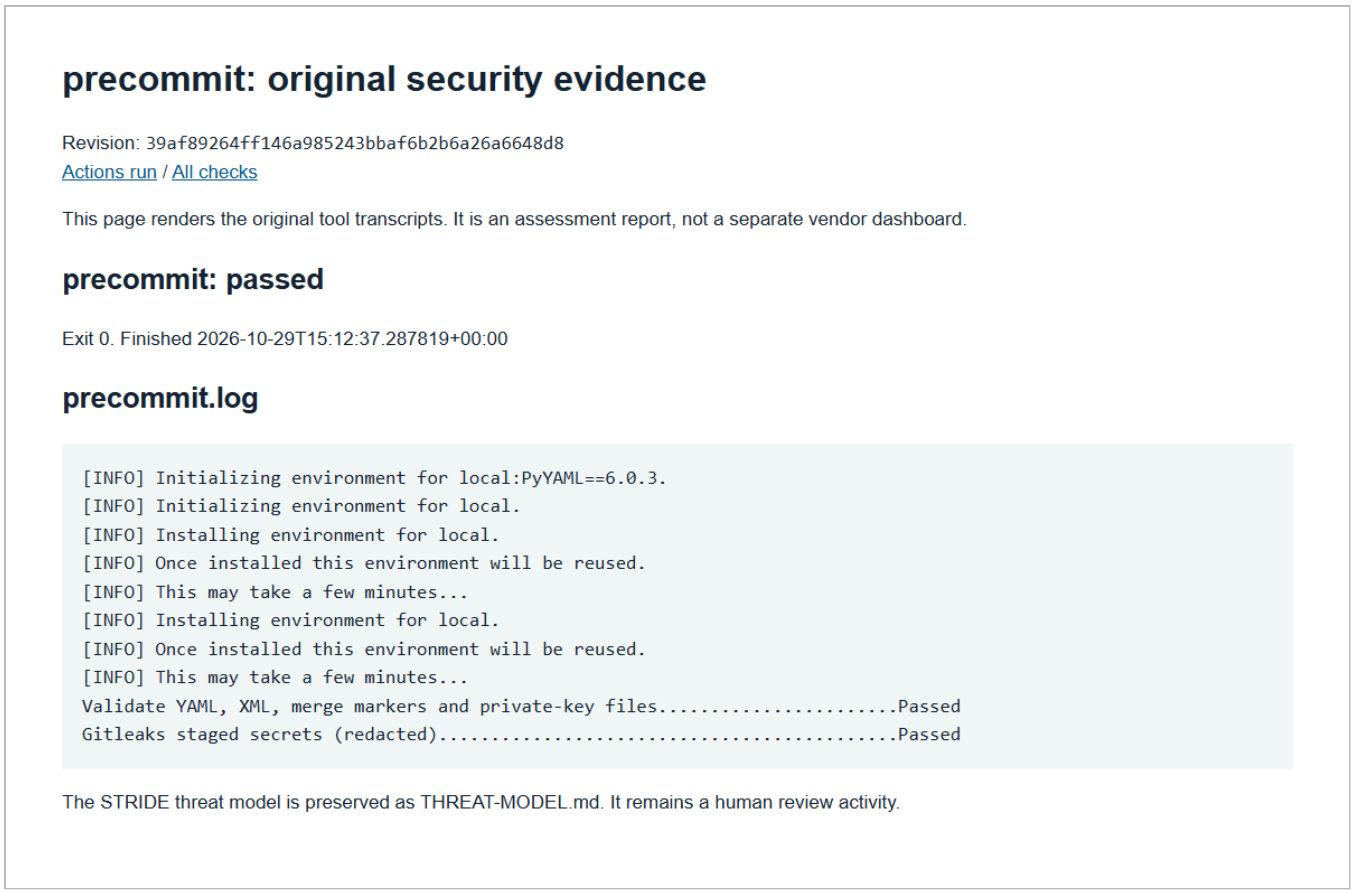
Stage by stage, with proof

A. Commit and pre-build security (before Maven compiles anything)

1. Threat modelling [Verified in a real run]

Technique: STRIDE threat model, reviewed by a human before coding
Tool: documentation/THREAT-MODEL.md
Blocks the pipeline when: The document must exist; CI preserves it with the run's reports
Report: `precommit/THREAT-MODEL.md`

Threat modelling is a human review: a green job proves the file is preserved, not that the analysis is complete.



precommit: original security evidence

Revision: `39af89264ff146a985243bbaf6b2b6a26a6648d8`
Actions run / All checks

This page renders the original tool transcripts. It is an assessment report, not a separate vendor dashboard.

precommit: passed

Exit 0. Finished 2026-10-29T15:12:37.287819+00:00

precommit.log

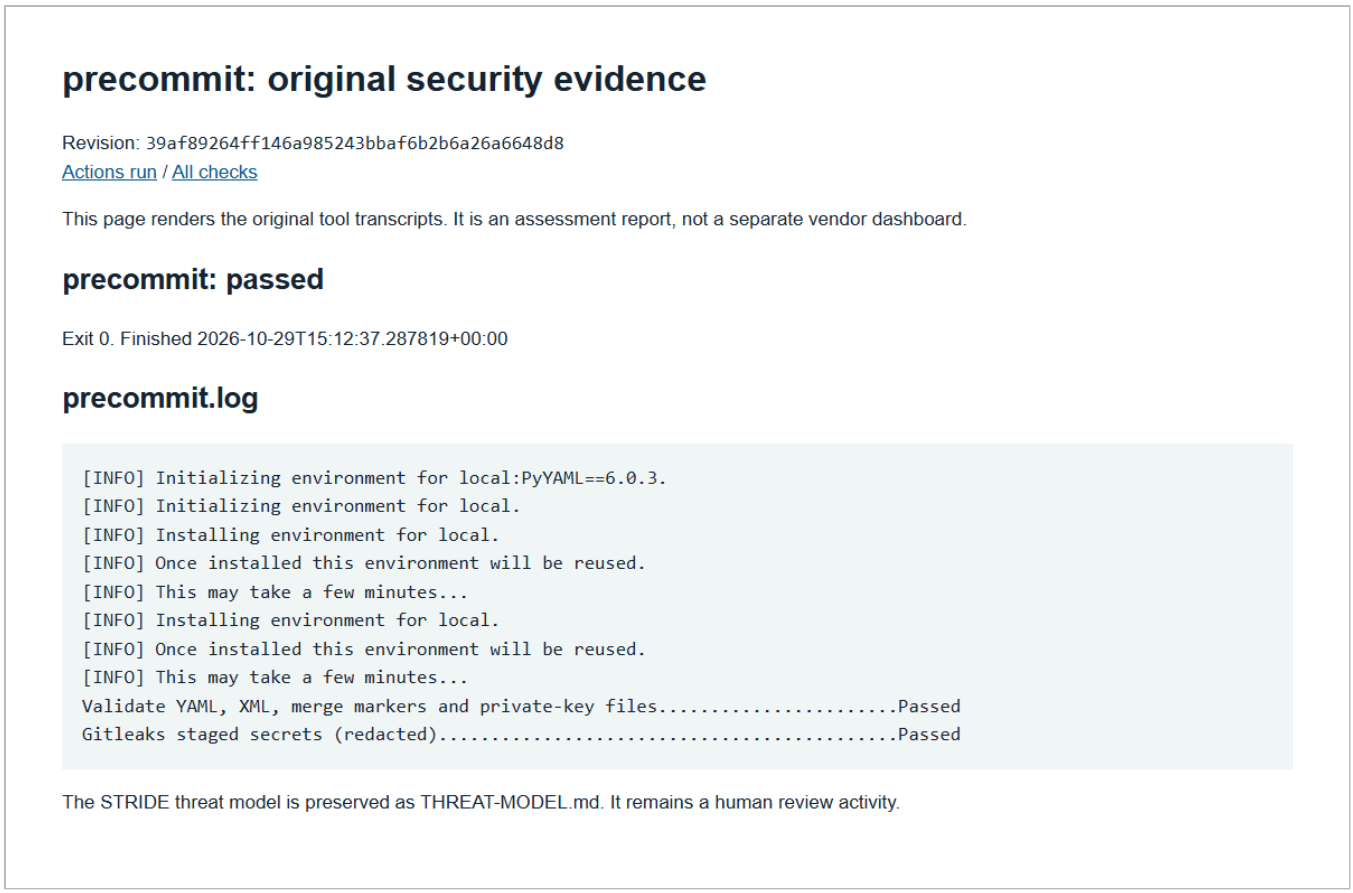
```
[INFO] Initializing environment for local:PyYAML==6.0.3.
[INFO] Initializing environment for local.
[INFO] Installing environment for local.
[INFO] Once installed this environment will be reused.
[INFO] This may take a few minutes...
[INFO] Installing environment for local.
[INFO] Once installed this environment will be reused.
[INFO] This may take a few minutes...
Validate YAML, XML, merge markers and private-key files.....................Passed
Gitleaks staged secrets (redacted)..........................................Passed
```

The STRIDE threat model is preserved as THREAT-MODEL.md. It remains a human review activity.

Pre-commit report page that preserves the threat model (run 40899036172, 29 Oct 2026)

2. Pre-commit hooks [Verified in a real run]

Technique: Local hooks replayed in CI: YAML/XML validity, merge markers, private-key headers, staged-secret scan
Tool: pre-commit, Gitleaks (staged snapshot)
Blocks the pipeline when: Any hook failure stops the pipeline
Report: `precommit/report.html`



precommit: original security evidence

Revision: `39af89264ff146a985243bbaf6b2b6a26a6648d8`
Actions run / All checks

This page renders the original tool transcripts. It is an assessment report, not a separate vendor dashboard.

precommit: passed

Exit 0. Finished 2026-10-29T15:12:37.287819+00:00

precommit.log

```
[INFO] Initializing environment for local:PyYAML==6.0.3.
[INFO] Initializing environment for local.
[INFO] Installing environment for local.
[INFO] Once installed this environment will be reused.
[INFO] This may take a few minutes...
[INFO] Installing environment for local.
[INFO] Once installed this environment will be reused.
[INFO] This may take a few minutes...
Validate YAML, XML, merge markers and private-key files.....................Passed
Gitleaks staged secrets (redacted)..........................................Passed
```

The STRIDE threat model is preserved as THREAT-MODEL.md. It remains a human review activity.

Hook transcript with Passed results (run 40899036172)

3. Static application security testing [Verified in a real run]

Technique: SAST with project rules for command execution, concatenated SQL and disabled TLS verification
Tool: Semgrep
Blocks the pipeline when: Any finding fails (--error --strict)
Report: `sast/report.html`

Deliberately narrow rule set, complemented by SonarQube.

sast: original security evidence

Revision: `39af89264ff146a985243bbaf6b2b6a26a6648d8`
Actions run / All checks

This page renders the original tool transcripts. It is an assessment report, not a separate vendor dashboard.

semgrep: passed

Exit 0. Finished 2026-10-29T15:13:05.819930+00:00

semgrep.log

```
┌──────────────┐
│ Scan Status │
└──────────────┘
  Scanning 19 files tracked by git with 3 Code rules:
  Scanning 19 files with 3 java rules.


┌──────────────┐
│ Scan Summary │
└──────────────┘
✅ Scan completed successfully.
 • Findings: 0 (0 blocking)
 • Rules run: 3
 • Targets scanned: 19
 • Parsed lines: ~100.0%
 • Scan was limited to files tracked by git
 • For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 3 rules on 19 files: 0 findings.
If Semgrep missed a finding, please send us feedback to let us know!
See https://semgrep.dev/docs/reporting-false-negatives/
```

19 files scanned, 3 rules, 0 findings (run 40899036172)

4. Secret scanning [Verified in a real run]

Technique: Full Git history scan with redacted output
Tool: Gitleaks
Blocks the pipeline when: Any new finding fails; one exact historical fingerprint is documented in SECURITY.md
Report: `secrets/report.html`

secrets: original security evidence

Revision: `39af89264ff146a985243bbaf6b2b6a26a6648d8`
Actions run / All checks

This page renders the original tool transcripts. It is an assessment report, not a separate vendor dashboard.

gitleaks: passed

Exit 0. Finished 2026-10-29T15:12:47.119750+00:00

gitleaks.log

```
    ○
    │\
    │ ○
    ○ ░
    ░    gitleaks

3:12PM INF 52 commits scanned.
3:12PM INF scanned ~535518 bytes (535.52 KB) in 386ms
3:12PM INF no leaks found
```

Gitleaks history scan passed (run 40899036172)

5. Dependency analysis and SBOM [Verified in a real run]

Technique: Software composition analysis of the source POM with a CycloneDX inventory
Tool: Trivy
Blocks the pipeline when: HIGH/CRITICAL fails; empty or unversioned inventory is rejected
Report: `dependencies/report.html`

An earlier green run had an empty inventory; the validator now rejects that.

dependencies: original security evidence

Revision: 39af89264ff146a985243bbaf6b2b6a26a6648d8
Actions run / All checks

This page renders the original tool transcripts. It is an assessment report, not a separate vendor dashboard.

Verified inventory


```
{
  "source": "pom.xml before Maven compilation",
  "pom_sha256": "ab31fea1a4a3c7cd4b3b299eeed74af0b58f40b19bc330574f02f231f2d90bca",
  "packages_analyzed": 108,
  "sbom_components": 109,
  "high_critical_findings": 0,
  "scanner_exit_code": 0,
  "scope": "Trivy Maven compile/runtime/import scopes; test and optional dependencies excluded"
}
```


dependencies: passed

Exit 0. Finished 2026-10-29T15:13:05.628222+00:00

dependency-resolution: passed

Exit 0. Finished 2026-10-29T15:12:59.400001+00:00

dependencies.log


```
2026-10-29T15:13:01Z    INFO    [vulndb] Need to update DB
2026-10-29T15:13:01Z    INFO    [vulndb] Downloading vulnerability DB...
2026-10-29T15:13:01Z    INFO    [vulndb] Downloading artifact...        repo="mirror.gcr.io/aquasec/trivy-db:2"
2026-10-29T15:13:05Z    INFO    [vulndb] Artifact successfully downloaded
repo="mirror.gcr.io/aquasec/trivy-db:2"
2026-10-29T15:13:05Z    INFO    [vuln] Vulnerability scanning is enabled
2026-10-29T15:13:05Z    INFO    Number of language-specific files       num=1
2026-10-29T15:13:05Z    INFO    [pom] Detecting vulnerabilities...

📣 Notices:
  - Version 0.76.1 of Trivy is now available, current version is 0.76.0

To suppress version checks, run Trivy scans with the --skip-version-check flag

2026-10-29T15:13:05Z    INFO    "--format cyclonedx" disables security scanning. Specify "--scanners vuln"
explicitly if you want to include vulnerabilities in the "cyclonedx" report.
{
  "source": "pom.xml before Maven compilation",
  "pom_sha256": "ab31fea1a4a3c7cd4b3b299eeed74af0b58f40b19bc330574f02f231f2d90bca",
  "packages_analyzed": 108,
  "sbom_components": 109,
  "high_critical_findings": 0,
  "scanner_exit_code": 0,
  "scope": "Trivy Maven compile/runtime/import scopes; test and optional dependencies excluded"
}
```


Verified inventory and SBOM summary (run 40899036172)

6. Infrastructure-as-code policy [Verified in a real run]

Technique: Terraform fmt/validate, policy scan, custom SSH-to-internet rule, workflow lint, Caddy config validation
Tool: Terraform, Checkov, actionlint
Blocks the pipeline when: Any unexcepted failure; ten exclusions are documented in SECURITY.md
Report: `iac/report.html`

iac: original security evidence

Revision: 39af89264ff146a985243bbaf6b2b6a26a6648d8
Actions run / All checks

This page renders the original tool transcripts. It is an assessment report, not a separate vendor dashboard.

Policy counts

- checkov-course.json: 43 passed, 0 failed, 0 skipped
- checkov-default.json: 27 passed, 0 failed, 0 skipped

checkov-course: passed

Exit 0. Finished 2026-10-29T15:13:26.230894+00:00

checkov-default: passed

Exit 0. Finished 2026-10-29T15:13:23.519117+00:00

checkov-course.log

```
terraform scan results:

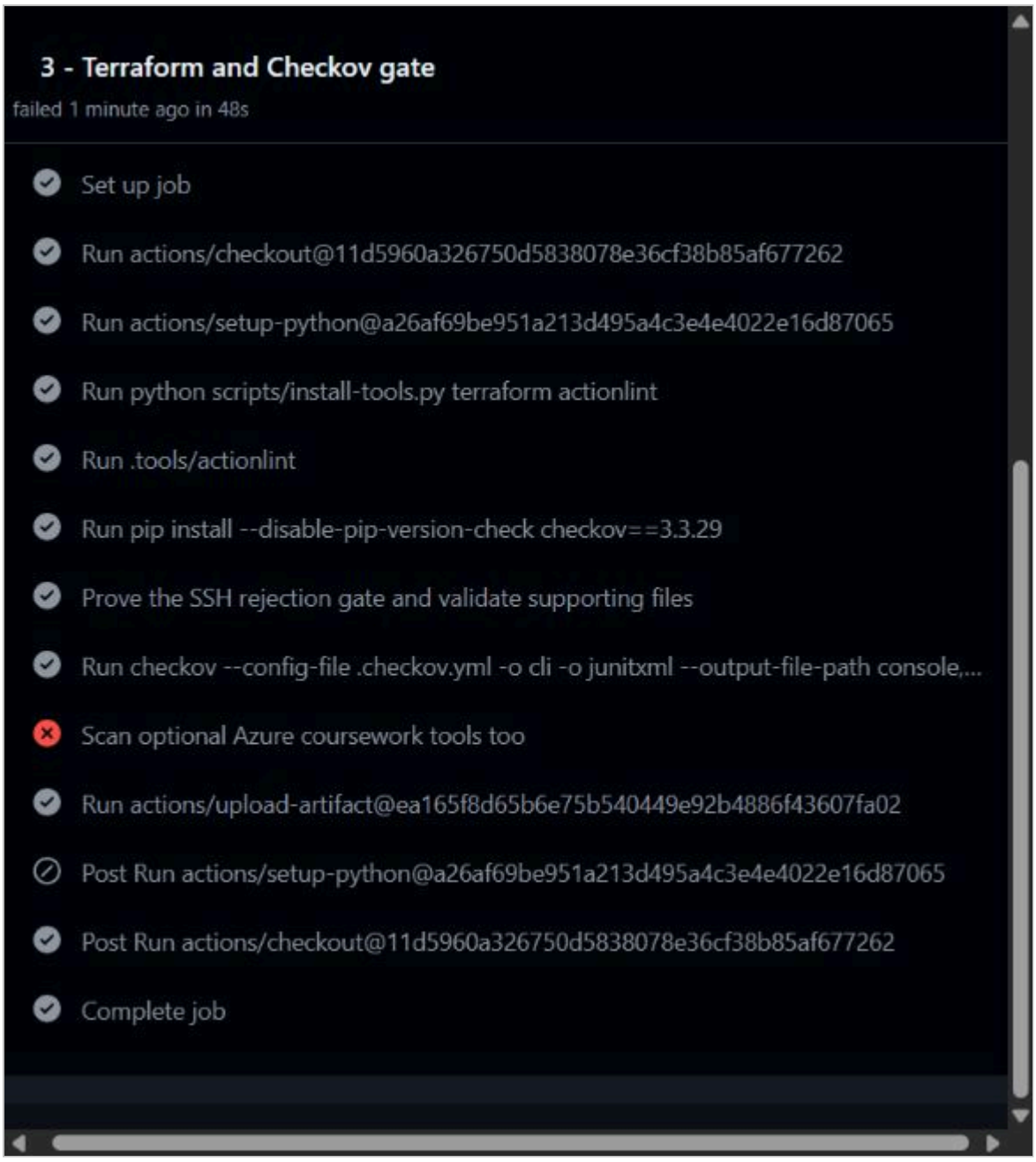
Passed checks: 43, Failed checks: 0, Skipped checks: 0
```

checkov-default.log

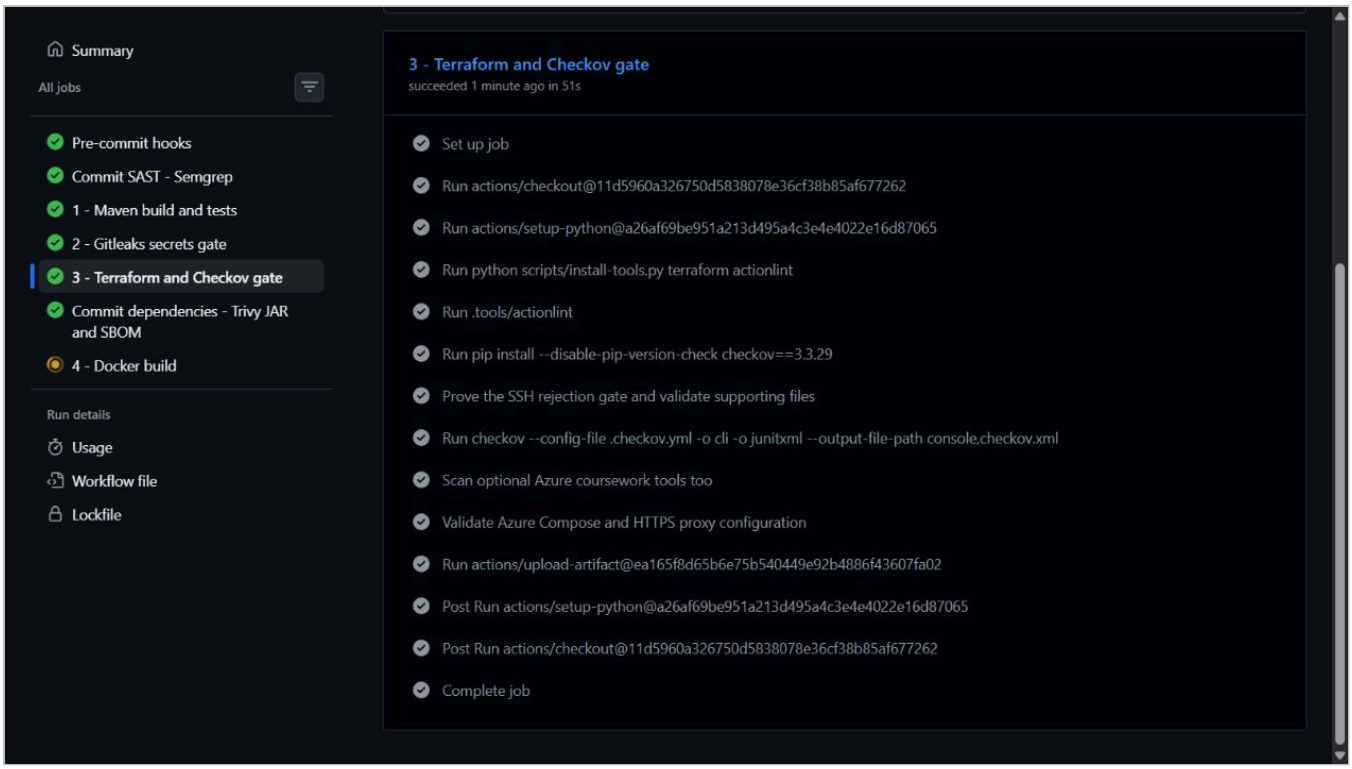
```
terraform scan results:

Passed checks: 27, Failed checks: 0, Skipped checks: 0
```

Checkov policy counts (run 40899036172)



Real finding CKV_AZURE_160 (public HTTP port 80) failing the gate (run 39805052147)



Passing rescan after the port was closed (run 39805591559)

7. Collect pre-build reports [Verified in a real run]

Technique: Aggregate every scanner output into one readable index
Tool: scripts/render-security-reports.py
Blocks the pipeline when: Runs even after a failed gate
Report: `index.html`

DevSecOps security reports

Report execution revision: `521e41e06bbb43163d35d8bc8b711c63993f796b`
Run: https://github.com/oussamakou/DEVOPS-Esprit-4NIDS6/actions/runs/42055245961

Order: checkout, local-hook replay and commit security gates before Maven compilation; Nmap acceptance after deployment. Each original record retains its own execution time and provenance. Threat modeling remains a human review activity.

Pipeline stages at a glance

Stage	Tool	Outcome	What it measured	Report
Pre-commit hooks and threat model	pre-commit, Gitleaks hook	**passed**	hook transcript and threat model	Open
Commit SAST	Semgrep	**passed**	Semgrep: 19 files scanned, 0 application findings, 0 scanner errors	Open
Commit secrets scan	Gitleaks	**passed**	full available Git history	Open
Source dependency scan and SBOM	Trivy, CycloneDX	**passed**	108 packages analyzed	Open
Infrastructure policy	Terraform, Checkov	**passed**	checkov-course.json: 43 passed, 0 failed, 0 skipped; checkov-default.json: 27 passed, 0 failed, 0 skipped	Open
Compile, test and coverage	Maven, Surefire, JaCoCo	**passed**	Surefire: 6 tests in 3 classes, 0 failures, 0 errors, 0 skipped; JaCoCo line coverage: 41.5% (17/41)	Open
Code quality gate	SonarQube	not run in this snapshot	-	-
Artifact repository	Nexus	not run in this snapshot	-	-
Image build and vulnerability gate	Docker, k3d smoke test, Trivy	**passed**	Trivy image scan of /tmp/image.tar: 0 CRITICAL, 0 HIGH findings across 2 scanned targets	Open
Image registry	Azure Container Registry	not run in this snapshot	-	-
Deployment	Docker Compose through Azure Run Command	not run in this snapshot	-	-
Network acceptance	Nmap	not run in this snapshot	-	-
Monitoring verification	API request, image and scrape-target check	not run in this snapshot	-	-
Metrics collection	Prometheus, Node Exporter	not run in this snapshot	-	-
Dashboards	Grafana	not run in this snapshot	-	-

Grafana (`http://127.0.0.1:3000/d/student-devsecops`, `/d/student-vms`) and Prometheus (`http://127.0.0.1:9091/targets`) are private: open them through the owner's SSH tunnel described in documentation/MONITORING-DEMO.md.

Security job outcomes

- security: success
- build: success
- docker: success
- trivy: success

Scanner measurements

- checkov-course.json: 43 passed, 0 failed, 0 skipped
- checkov-default.json: 27 passed, 0 failed, 0 skipped
- Semgrep: 19 files scanned, 0 application findings, 0 scanner errors

- Source dependency packages analyzed: 108

Combined index generated by CI (run 42055245961, pull request 25)

B. Build, quality and artifact

8. Compile, test and coverage [Verified in a real run]

Technique: Unit and HTTP integration tests with coverage measurement
Tool: Maven, JUnit (Surefire), JaCoCo
Blocks the pipeline when: Any failed test blocks the image build
Report: `build/report.html`

6 tests, 41.5% line coverage in the CI run shown.

Compile, test and coverage: original evidence

Tool: Maven, Surefire, JaCoCo
Revision: `521e41e06bbb43163d35d8bc8b711c63993f796b`
Actions run / All stages

This page renders the original tool transcripts and recorded facts. It is an assessment report, not a separate vendor dashboard.

Measured by this stage

- Surefire: 6 tests in 3 classes, 0 failures, 0 errors, 0 skipped
- JaCoCo line coverage: 41.5% (17/41)

maven: passed

Exit 0. Started 2026-11-05T15:29:57.804310+00:00; finished 2026-11-05T15:30:26.168951+00:00

maven.log

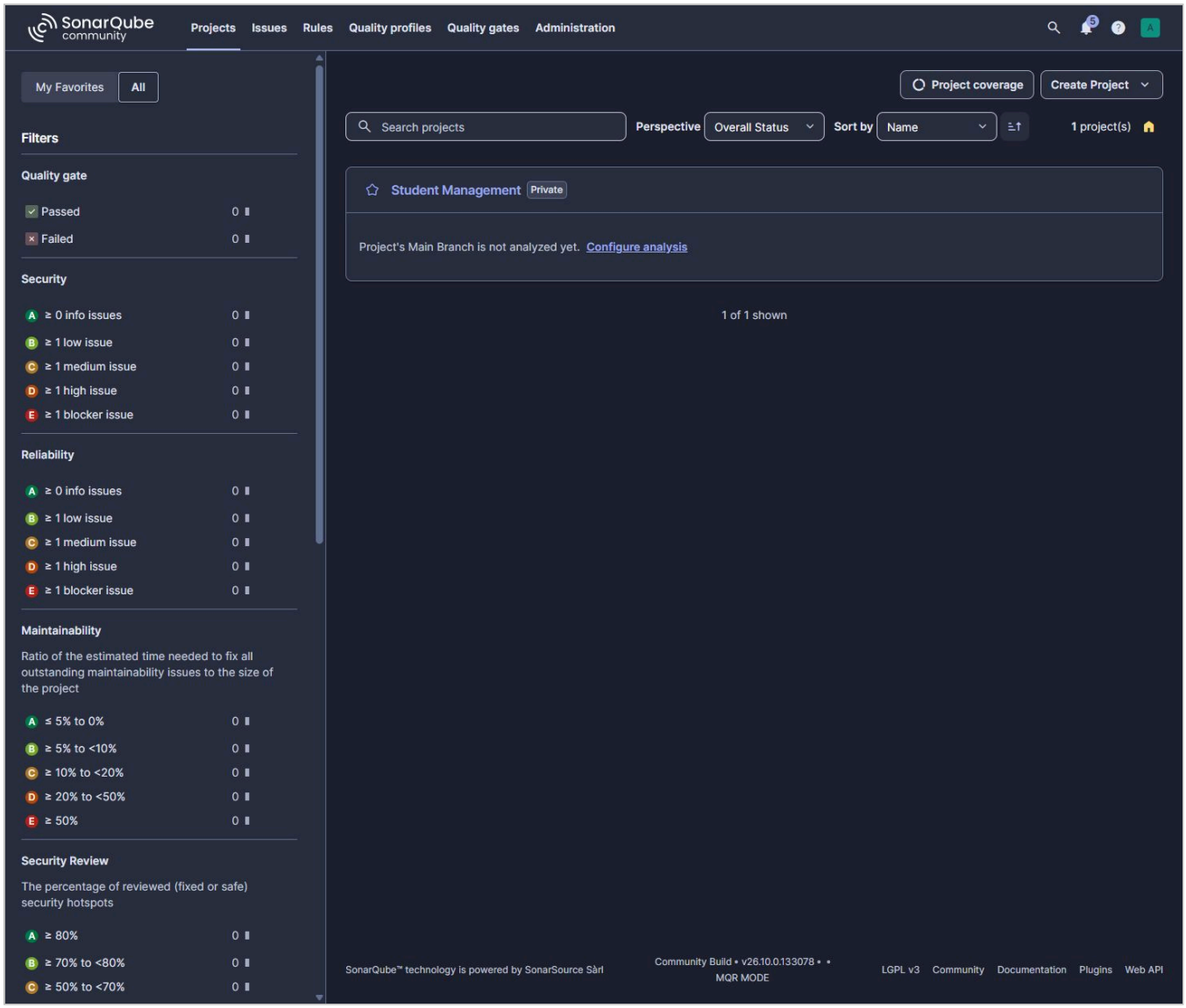
```
[INFO] Scanning for projects...
[INFO]
[INFO] --------------------< tn.esprit:student-management >--------------------
[INFO] Building student-management 0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- jacoco:0.8.11:prepare-agent (prepare-agent) @ student-management ---
[INFO] argLine set to -javaagent:/home/runner/.m2/repository/org/jacoco/org.jacoco.agent/0.8.11/org.jacoco.agent-
0.8.11-runtime.jar=destfile=/home/runner/work/DEVOPS-Esprit-4NIDS6/DEVOPS-Esprit-4NIDS6/target/jacoco.exec
[INFO]
[INFO] --- resources:3.5.0:resources (default-resources) @ student-management ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.15.0:compile (default-compile) @ student-management ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 19 source files with javac [debug parameters release 17] to target/classes
[INFO]
[INFO] --- resources:3.5.0:testResources (default-testResources) @ student-management ---
[INFO] Copying 1 resource from src/test/resources to target/test-classes
[INFO]
[INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ student-management ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 3 source files with javac [debug parameters release 17] to target/test-classes
[INFO]
[INFO] --- surefire:3.5.6:test (default-test) @ student-management ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running tn.esprit.studentmanagement.StudentManagementApplicationTests
15:30:10.743 [main] INFO org.springframework.test.context.support.AnnotationConfigContextLoaderUtils -- Could not
detect default configuration classes for test class
[tn.esprit.studentmanagement.StudentManagementApplicationTests]: StudentManagementApplicationTests does not
declare any static, non-private, non-final, nested classes annotated with @Configuration.
15:30:10.960 [main] INFO org.springframework.boot.test.context.SpringBootTestContextBootstrapper -- Found
@SpringBootConfiguration tn.esprit.studentmanagement.StudentManagementApplication for test class
tn.esprit.studentmanagement.StudentManagementApplicationTests
15:30:11.227 [main] INFO org.springframework.test.context.support.AnnotationConfigContextLoaderUtils -- Could not
detect default configuration classes for test class
[tn.esprit.studentmanagement.StudentManagementApplicationTests]: StudentManagementApplicationTests does not
declare any static, non-private, non-final, nested classes annotated with @Configuration.
```

Maven stage page: log, test counts and coverage (run 42055245961)

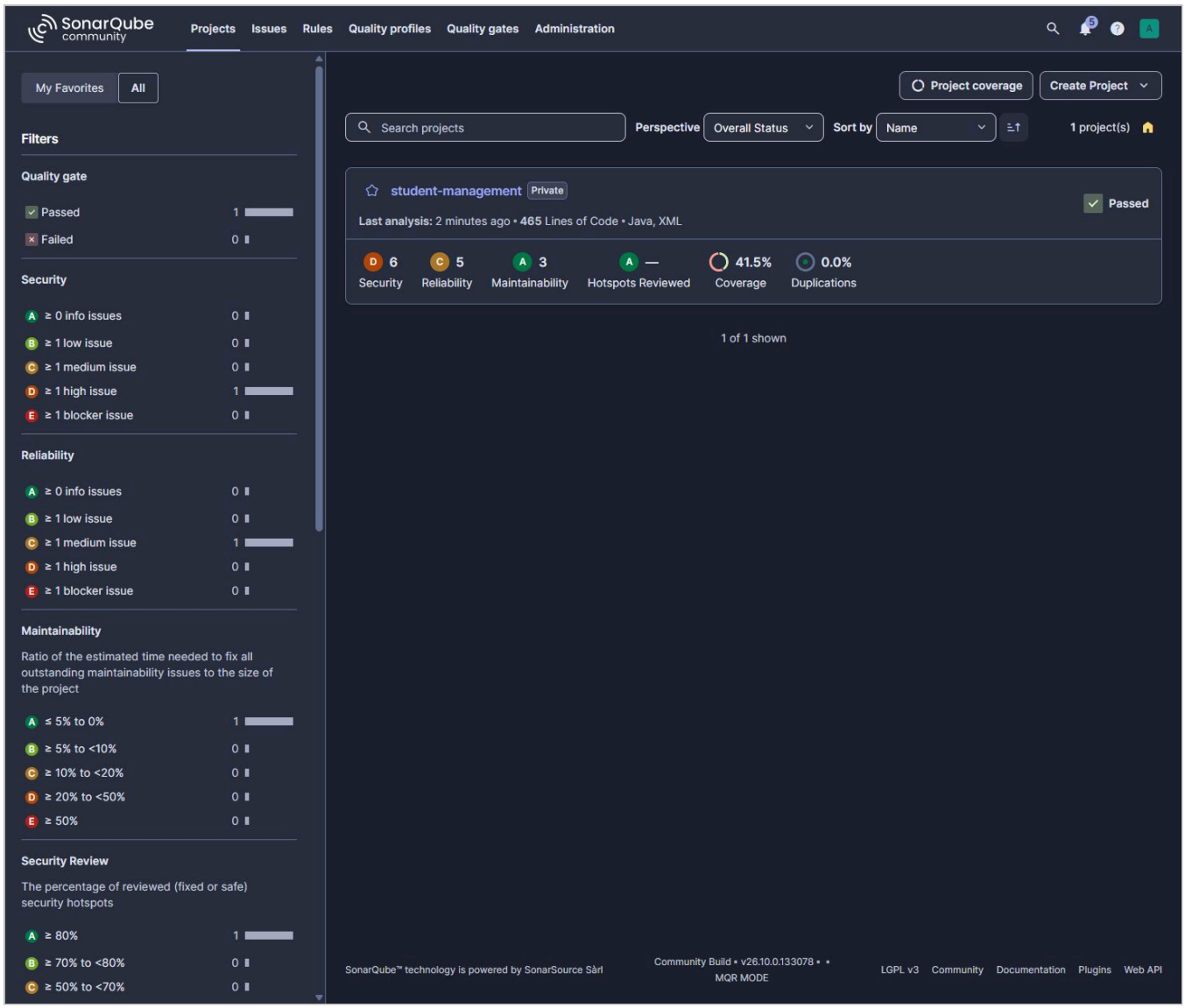
9. Code quality gate [Partly verified]

Technique: Static analysis with a blocking quality gate
Tool: SonarQube Community Build
Blocks the pipeline when: The pipeline waits for the gate and fails if it fails
Report: `quality/report.html`

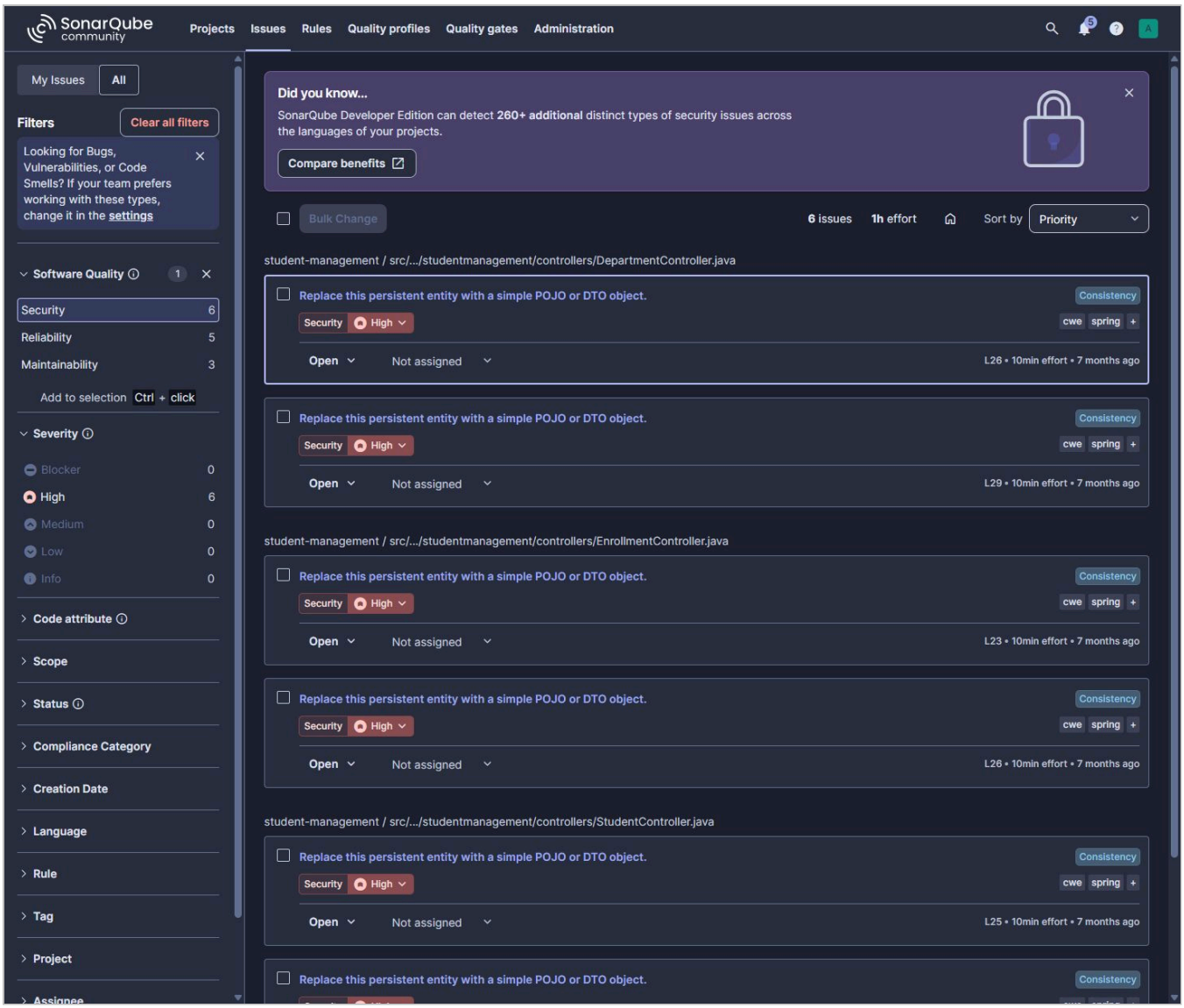
First baseline: gate OK but 11 open issues, including six HIGH entity-binding findings that are not yet fixed. The new quality report page has not run live yet.



Private project ready before the first analysis



First baseline: 41.5% coverage and 11 issues (run 39883066388)

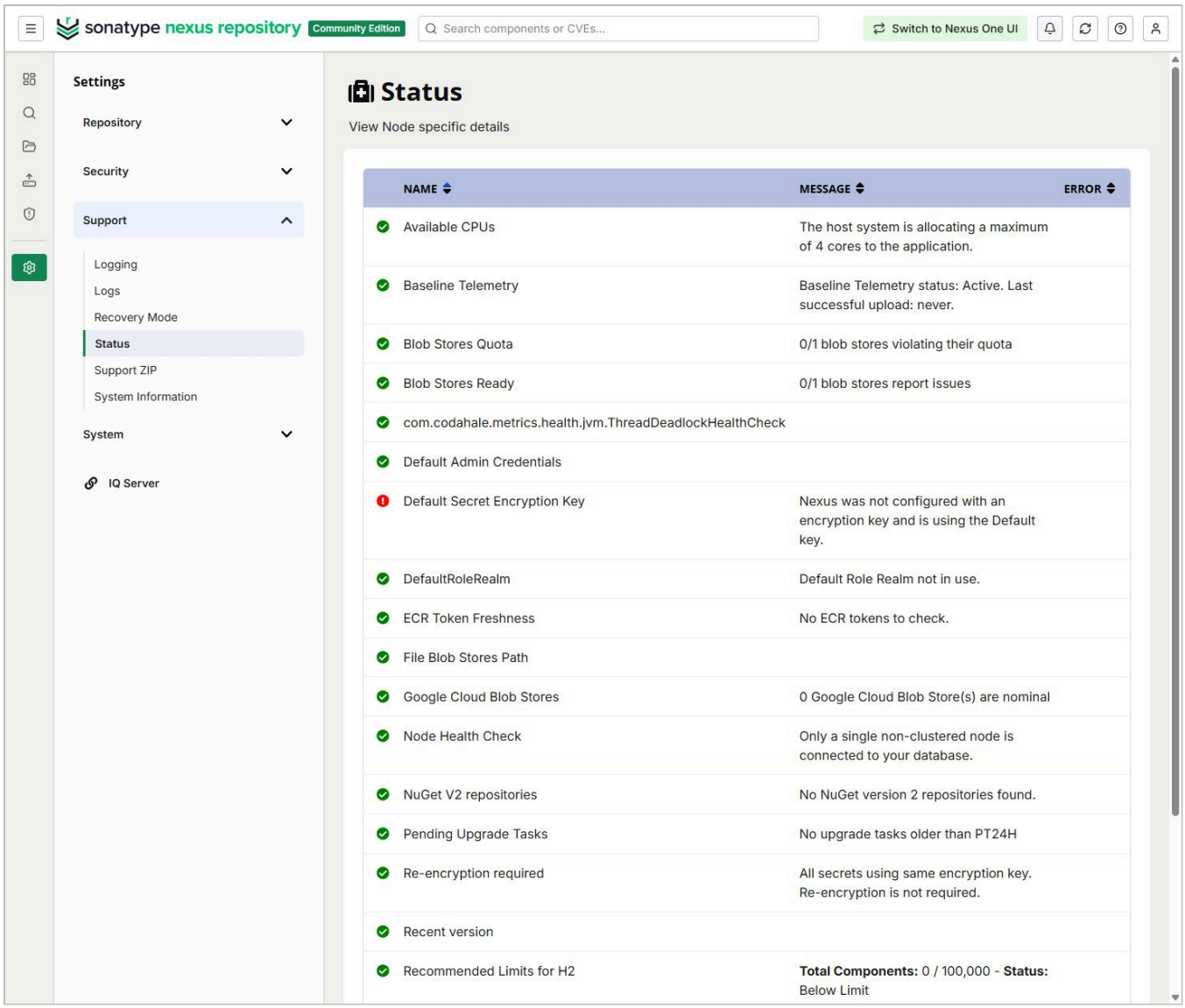


Six HIGH java:S4684 findings in the controllers

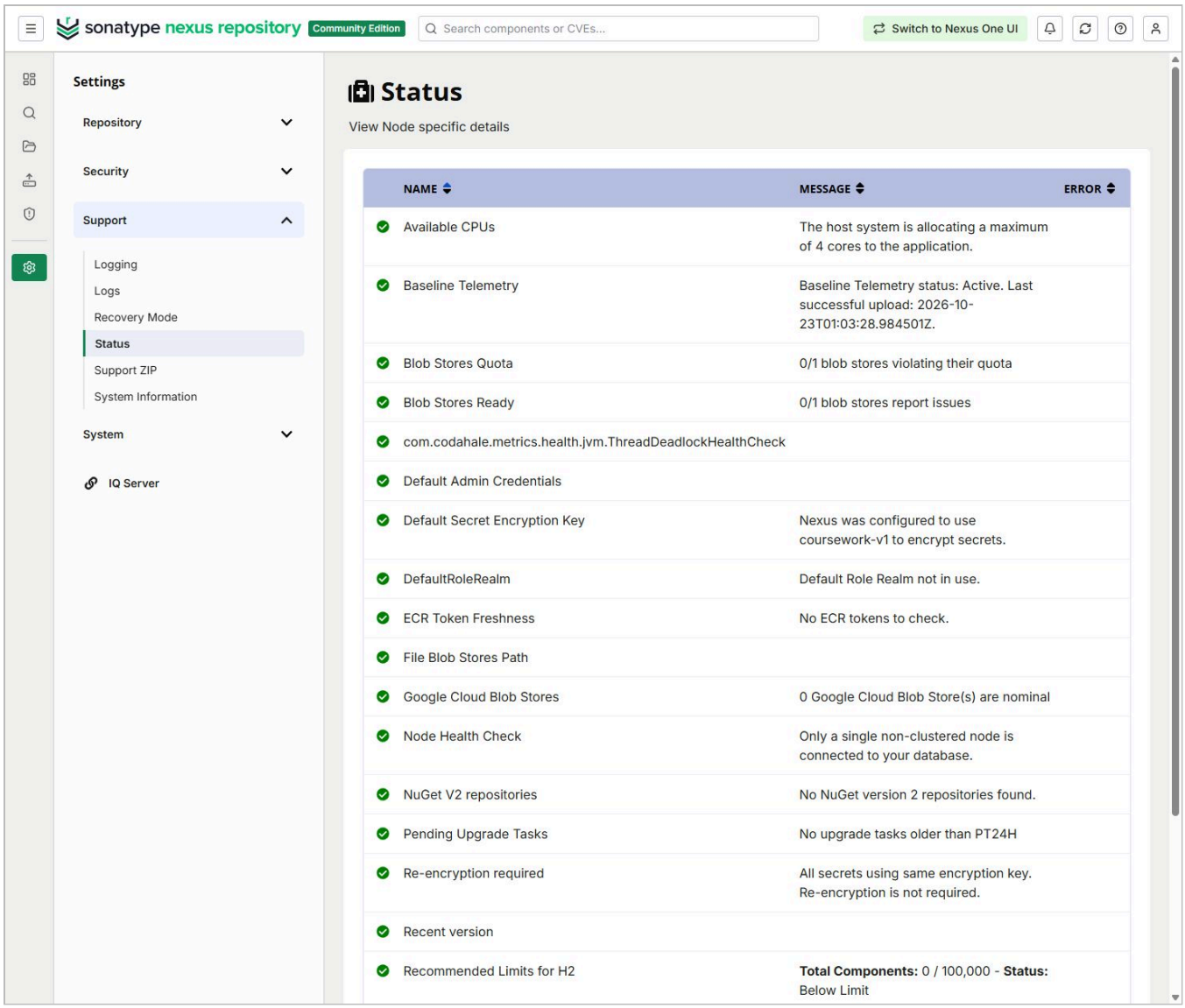
10. Artifact repository [Partly verified]

Technique: Publish the tested JAR, then compare the stored checksum with the tested JAR
Tool: Sonatype Nexus
Blocks the pipeline when: A different stored checksum fails the stage
Report: `artifact/report.html`

Upload has run live. The checksum verification is implemented and tested against a fake repository but has not run against the live Nexus yet.



Real finding: Nexus used the default encryption key



After remediation with a private key

11. Image build and vulnerability gate [Verified in a real run]

Technique: Build a non-root image from the tested JAR, smoke-test it on ephemeral k3s, scan the final image
Tool: Docker, k3d, Trivy
Blocks the pipeline when: Any HIGH/CRITICAL finding fails, including unfixed ones
Report: `image/report.html`

An earlier run was blocked by 11 HIGH/CRITICAL findings; see SECURITY-GATE-CASE-STUDY.md.

Image build and vulnerability gate: original evidence

Tool: Docker, k3d smoke test, Trivy
Revision: `521e41e06bbb43163d35d8bc8b711c63993f796b`
Actions run / All stages

This page renders the original tool transcripts and recorded facts. It is an assessment report, not a separate vendor dashboard.

Measured by this stage

- Trivy image scan of /tmp/image.tar: 0 CRITICAL, 0 HIGH findings across 2 scanned targets

Recorded facts

image	`student-management:521e41e06bbb43163d35d8bc8b711c63993f796b`
image_id	`sha256:9cddfc83da10fac8c12cf836f62131fad42eb5d614c188566fc535173d01b592`
size_bytes	`249414700`
tar_sha256	`9426a9ae1ed23d2dd9bec61f0f0d6cd069427cc49ca6ae759198baf5ad9bfca3`

k3s-smoke: passed

Exit 0. Started 2026-11-05T15:31:25.329234+00:00; finished 2026-11-05T15:33:41.592795+00:00

trivy-image: passed

Exit 0. Started 2026-11-05T15:34:30.499531+00:00; finished 2026-11-05T15:34:54.596603+00:00

k3s-smoke.log

```
INFO[0000] Prep: Network
INFO[0000] Created network 'k3d-student-ci'
INFO[0000] Created image volume k3d-student-ci-images
INFO[0000] Starting new tools node...
INFO[0000] Pulling image 'ghcr.io/k3d-io/k3d-tools:5.9.0'
INFO[0001] Creating node 'k3d-student-ci-server-0'
INFO[0001] Starting node 'k3d-student-ci-tools'
INFO[0001] Pulling image 'rancher/k3s:v1.37.1-k3s1'
INFO[0005] Creating LoadBalancer 'k3d-student-ci-serverlb'
INFO[0005] Pulling image 'ghcr.io/k3d-io/k3d-proxy:5.9.0'
INFO[0007] Using the k3d-tools node to gather environment information
INFO[0007] HostIP: using network gateway 172.18.0.1 address
INFO[0007] Starting cluster 'student-ci'
INFO[0007] Starting servers...
INFO[0007] Starting node 'k3d-student-ci-server-0'
INFO[0011] All agents already running.
INFO[0011] Starting helpers...
INFO[0011] Starting node 'k3d-student-ci-serverlb'
INFO[0017] Injecting records for hostAliases (incl. host.k3d.internal) and for 2 network members into CoreDNS
configmap...
INFO[0019] Cluster 'student-ci' created successfully!
INFO[0019] You can now use it like this:
kubectl cluster-info
```

Image stage page: build facts, k3s smoke test and Trivy result, 0 HIGH/CRITICAL (run 42055245961)

C. Delivery to Azure

12. Image registry [Partly verified]

Technique: Push the scanned image and deploy by immutable digest
Tool: Azure Container Registry, OIDC login
Blocks the pipeline when: The digest must belong to the pushed image
Report: `registry/report.html`

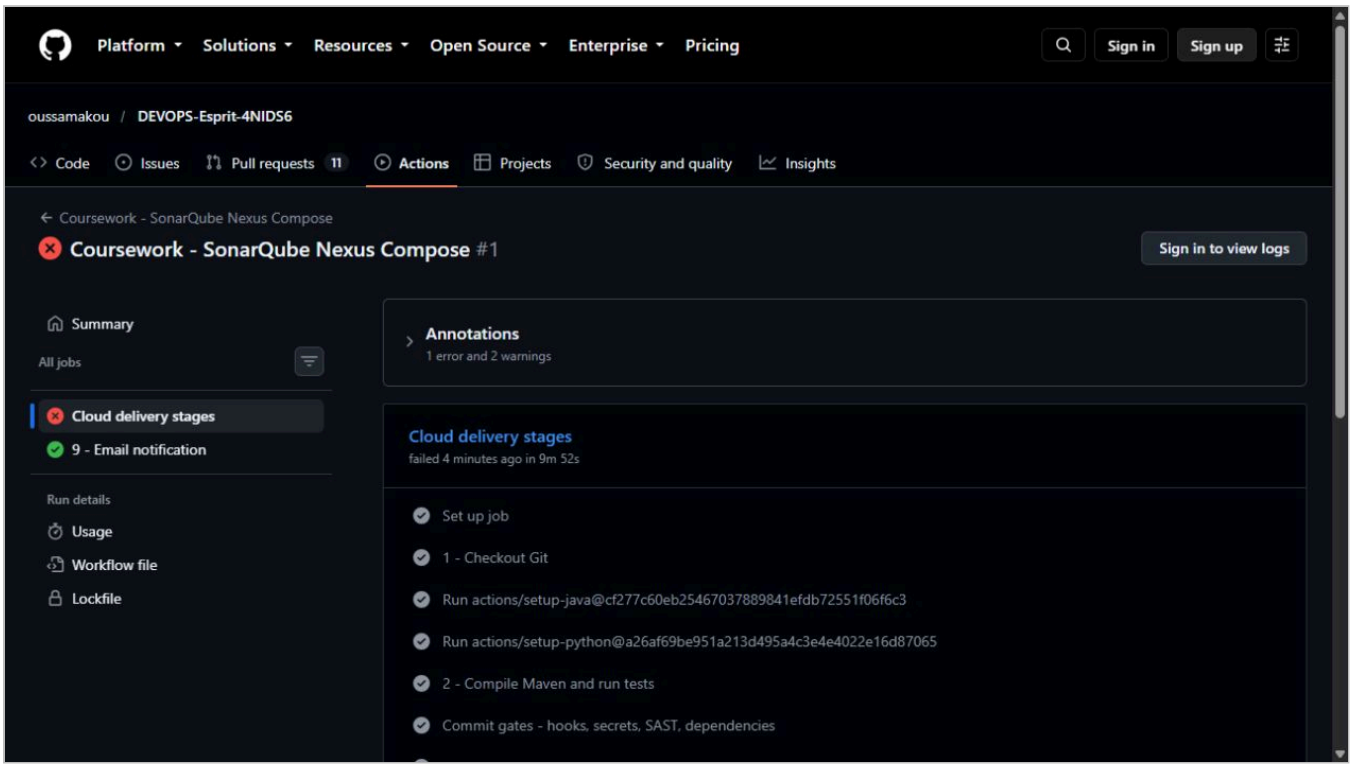
Pushes succeeded in earlier runs; the registry report page and a portal screenshot of the digest are pending.

No screenshot of this stage exists yet.

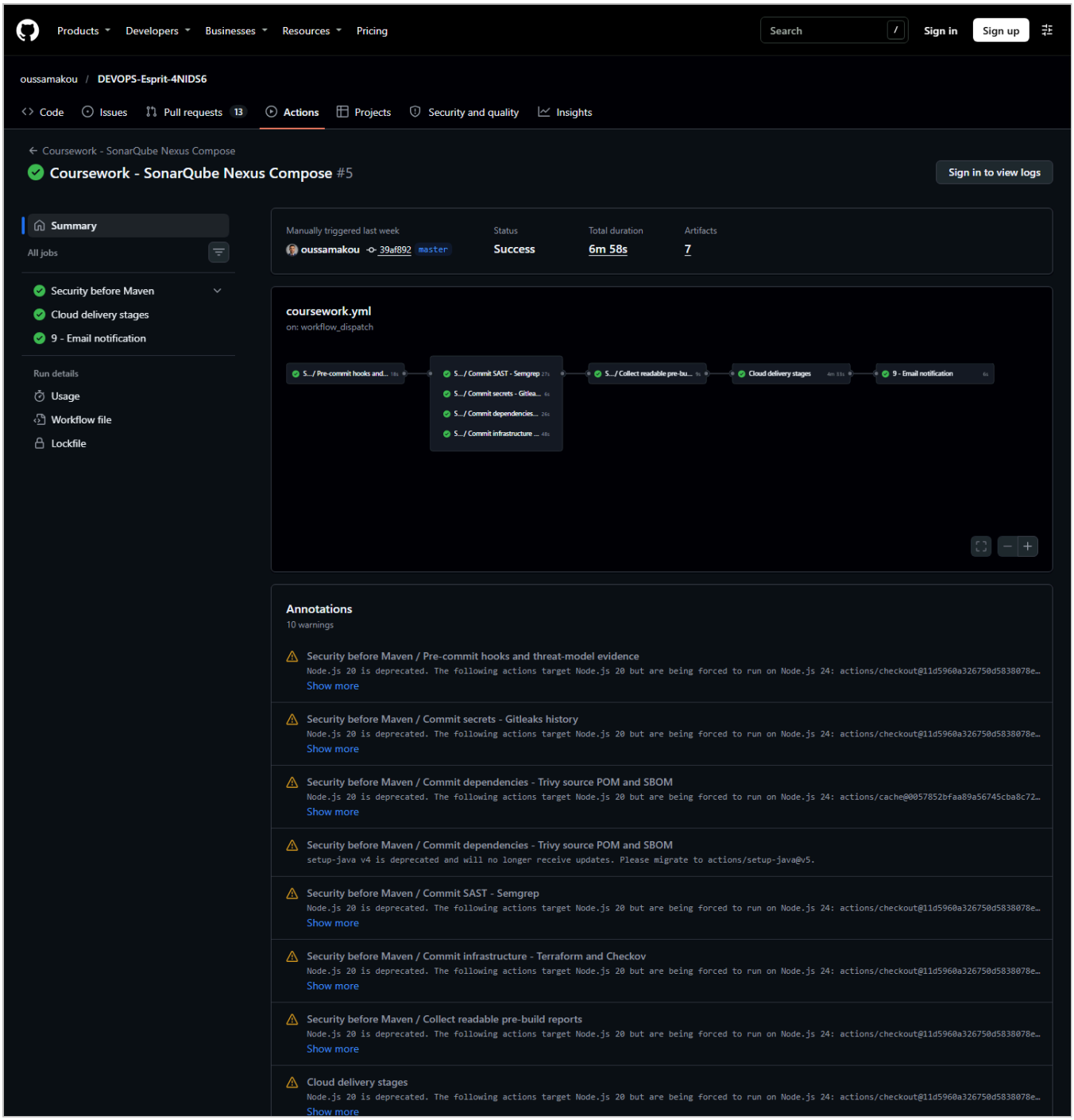
13. Deployment [Verified in a real run]

Technique: Docker Compose started on the app VM through Azure Run Command, pulling with the VM's managed identity
Tool: Azure Run Command, Docker Compose
Blocks the pipeline when: The guest must print DEPLOY_SUCCEEDED and a real database-backed API request must pass
Report: `deploy/report.html`

The first run failed here and was repaired (COMPOSE-STARTUP-CASE-STUDY.md).



Real first failure at the Compose stage (run 39883066388)



Full coursework run succeeding after the repair (run 40899036172)

D. Acceptance, monitoring and notification

14. Network acceptance [Verified in a real run]

Technique: Scoped TCP exposure test from outside and inside Azure against an allow-list
Tool: Nmap
Blocks the pipeline when: Unexpected open, missing required or incomplete scans fail
Report: `acceptance/report.html`

UDP and ports outside the policy list are not tested.

Nmap acceptance: PASSED

Owned studentcourse Azure lab. TCP connect scans with light service detection; no exploit scripts.

Coverage: 1-1024,2375,2376,3000,3306,5432,6443,8080,8081,8089,8443,9000,9090,9091,9100,10250,30089. Ports outside this list and UDP were not tested. Filtered means probes received no usable response from this viewpoint. A service version is an observation, not proof of a vulnerability.

External runner: only tools HTTPS ports 443/8443 may be open; SSH is owner-only. Internal application-VM scans verify the private exporters and localhost application/monitoring endpoints. Required open ports provide positive controls; incomplete scans and missing reports fail acceptance.

Public viewpoint: passed

app-public — 158.158.49.102 — passed

Open TCP ports: []; unexpected: []; missing required: []

Original reports: Nmap XML · Nmap text · console log

TCP port	State	Service observation

tools-public — 68.221.67.223 — passed

Open TCP ports: [443, 8443]; unexpected: []; missing required: []

Original reports: Nmap XML · Nmap text · console log

TCP port	State	Service observation
443	open	https
8443	open	https-alt

Private viewpoint: passed

app-loopback — 127.0.0.1 — passed

Open TCP ports: [22, 3000, 8089, 9091]; unexpected: []; missing required: []

Original reports: Nmap XML · Nmap text · console log

TCP port	State	Service observation
22	open	ssh OpenSSH 9.6p1 Ubuntu 3ubuntu13.20
3000	open	ppp
8089	open	unknown
9091	open	xmltec-xmlmail

app-private — 10.42.1.4 — passed

Open TCP ports: [22, 9100]; unexpected: []; missing required: []

Original reports: Nmap XML · Nmap text · console log

TCP port	State	Service observation
22	open	ssh OpenSSH 9.6p1 Ubuntu 3ubuntu13.20
9100	open	jetdirect

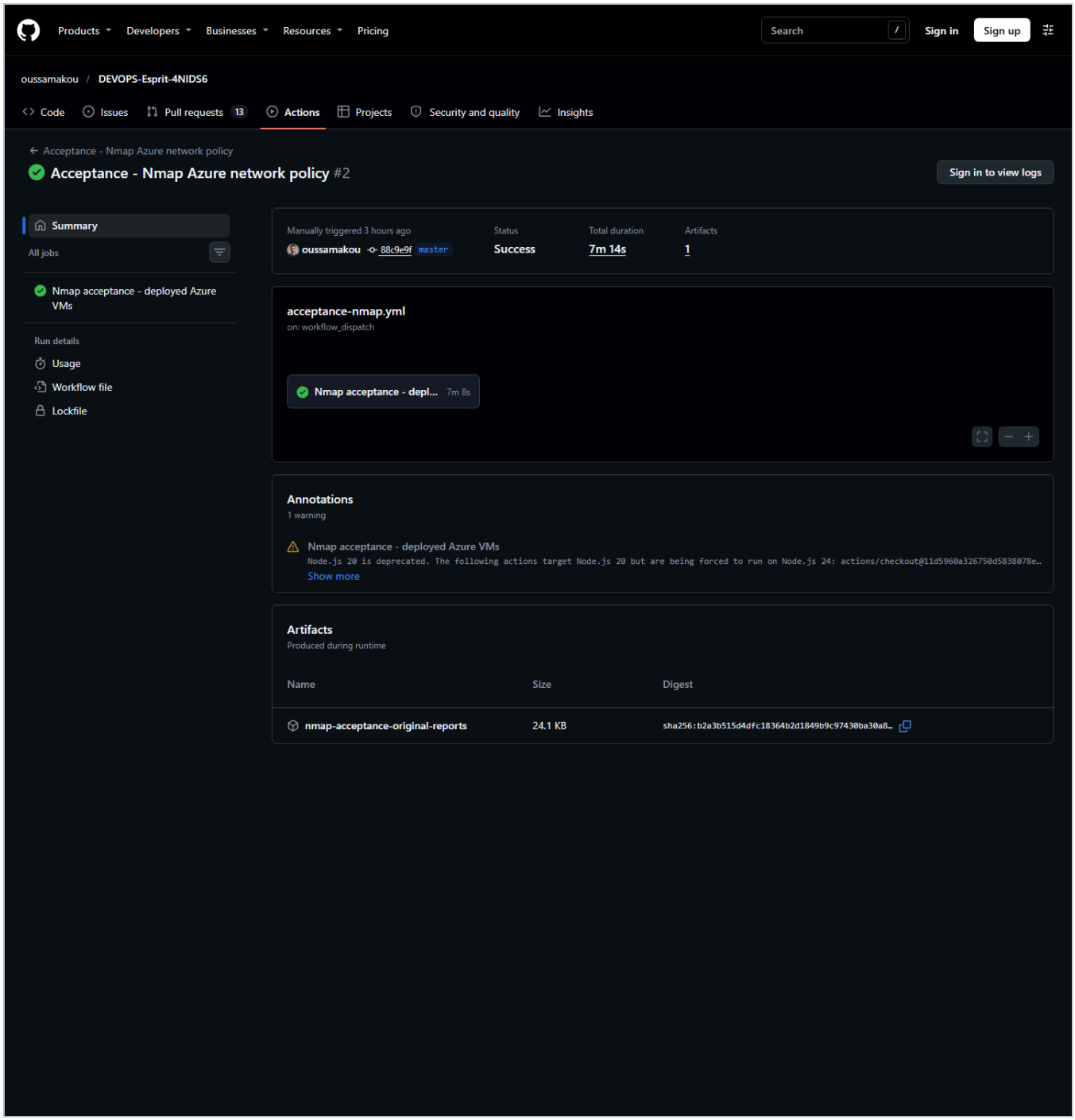
tools-private — 10.42.2.4 — passed

Open TCP ports: [22, 443, 8443, 9100]; unexpected: []; missing required: []

Original reports: Nmap XML · Nmap text · console log

TCP port	State	Service observation
22	open	ssh OpenSSH 9.6p1 Ubuntu 3ubuntu13.20
443	open	https
8443	open	https-alt
9100	open	jetdirect

Acceptance PASSED with public and private viewpoints (run 42016218929)



The Nmap acceptance workflow run (42016218929)

15. Monitoring: Prometheus and Grafana [Implemented, not yet run live]

Technique: Verify scrape targets, sample real queries after light API traffic, list provisioned dashboards
Tool: Prometheus, Node Exporter, Grafana
Blocks the pipeline when: API target and both host exporters UP, Grafana database healthy, both dashboards provisioned
Report: `monitoring/, prometheus/, grafana/ report.html`

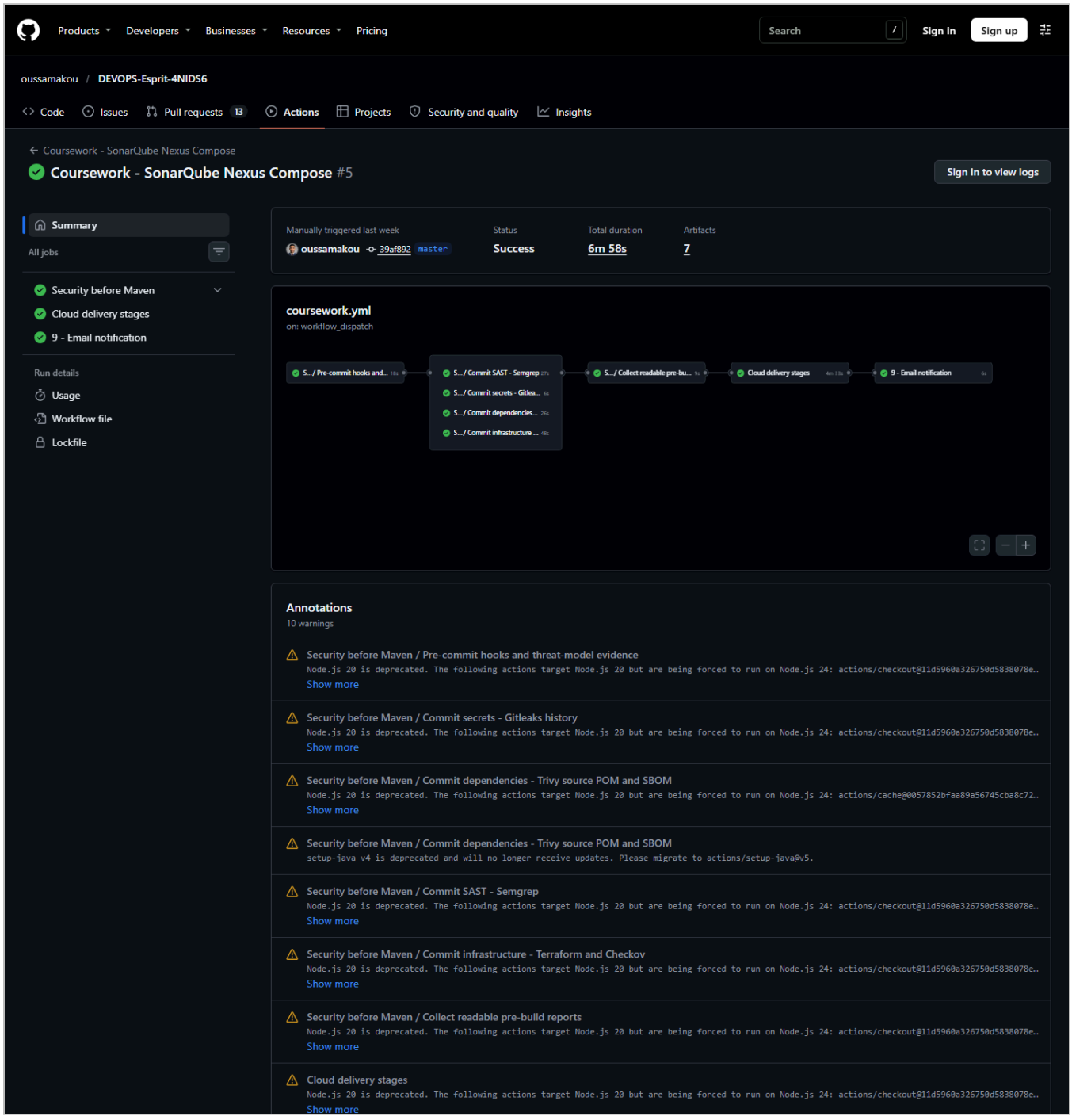
Target UP was verified on the VM earlier; the collection of evidence for the report is implemented and tested offline but has not run live. Grafana charts in the report are drawn from Prometheus data, not Grafana screenshots.

No screenshot of this stage exists yet.

16. Email notification [Partly verified]

Technique: Status, per-stage results and the run link sent at the end of every master run
Tool: SMTP over STARTTLS
Blocks the pipeline when: Runs even after failure or cancellation
Report: `the email, and the job in the run graph`

The job succeeded in every coursework run, shown in the graph. Receipt in the inbox is unconfirmed; the per-stage list in the message is new and has not run live.



Run graph ending with the green email job (run 40899036172)

Real failures found and fixed

- Trivy blocked the first image with 11 HIGH/CRITICAL findings; fixed by upgrading Spring Boot and Jackson (SECURITY-GATE-CASE-STUDY.md).
- Checkov rejected public HTTP port 80 (CKV_AZURE_160); the port was closed and the rescan passed (screenshots under Infrastructure-as-code policy).

- The first Compose deployment failed on a missing localhost port mapping and config file permissions; repaired and re-run (see Deployment).
- Nexus reported the default encryption key; replaced with a private key (see Artifact repository).
- SonarQube reported six HIGH entity-binding findings. They are recorded and still open.

Not yet proven (to capture)

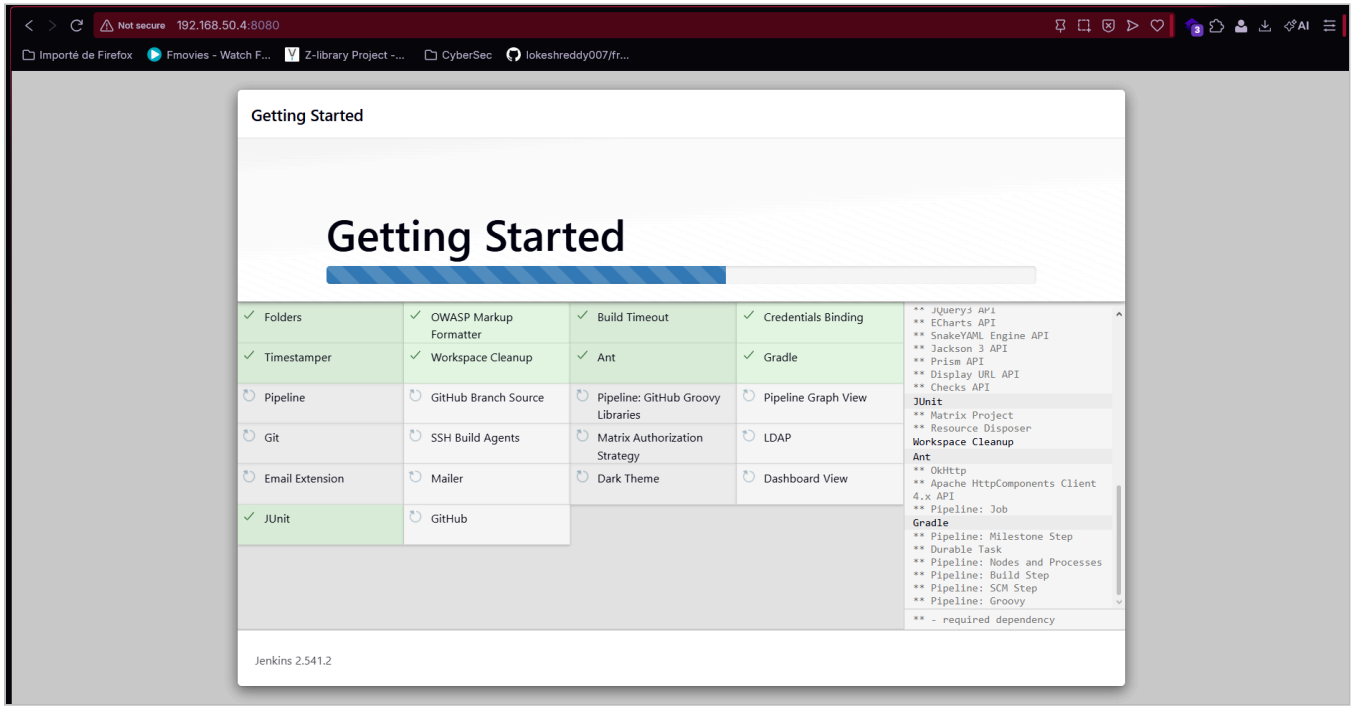
- SonarQube dashboard for a current analysis (needs sign-in)
- Nexus repository browser showing the stored JAR (needs sign-in)
- Azure portal registry page showing the image digest
- Prometheus targets page and both Grafana dashboards under traffic (needs the SSH tunnel and the Grafana password)
- The received status email in the inbox
- The new report pages for the Azure stages, after the first successful run on master

How to open everything

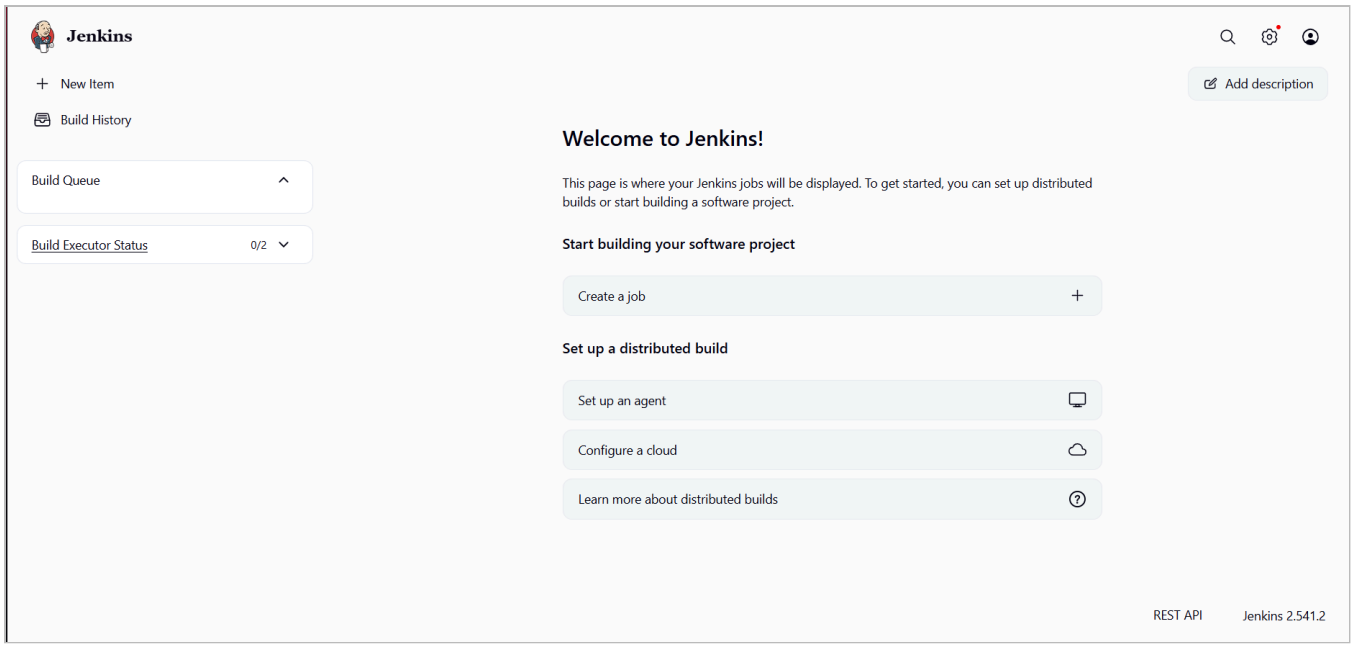
See documentation/PROFESSOR-DEMO.md: the Actions run and its artifacts, the SonarQube and Nexus links in the report, and the SSH tunnel for Prometheus (`http://127.0.0.1:9091/targets`) and Grafana (`http://127.0.0.1:3000/d/student-devsecops`).

Appendix: the original Jenkins and Minikube pipeline

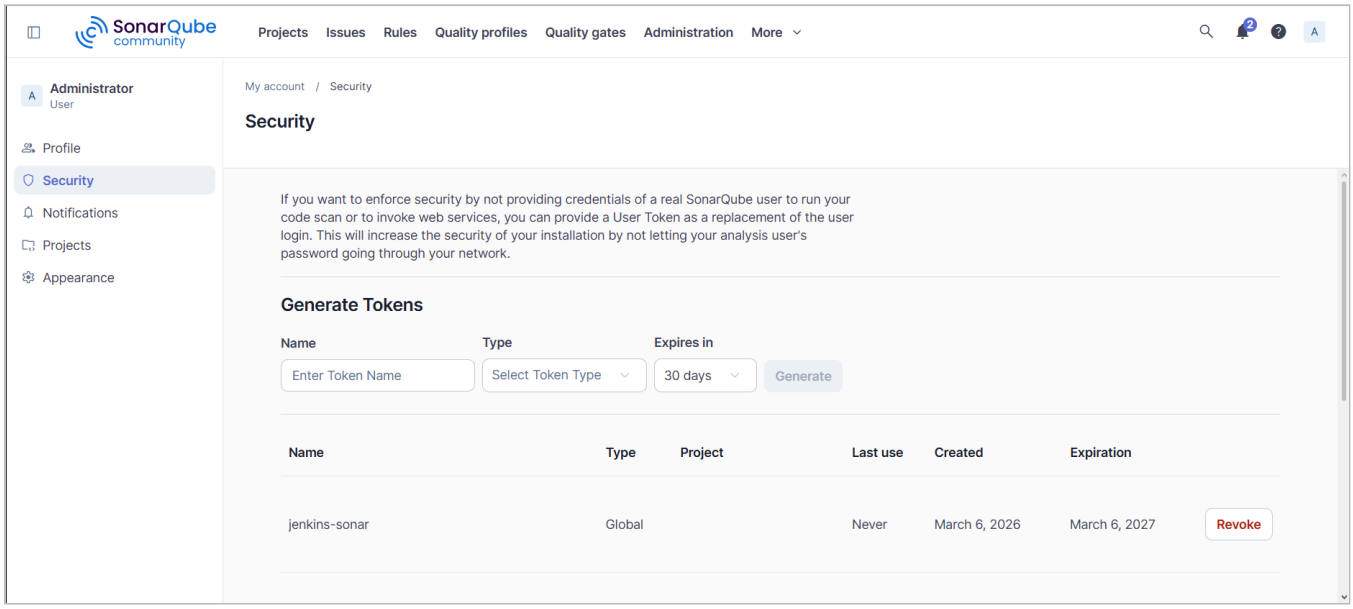
The project started with this school pipeline, preserved in documentation/LEGACY-JENKINS-MINIKUBE.md.



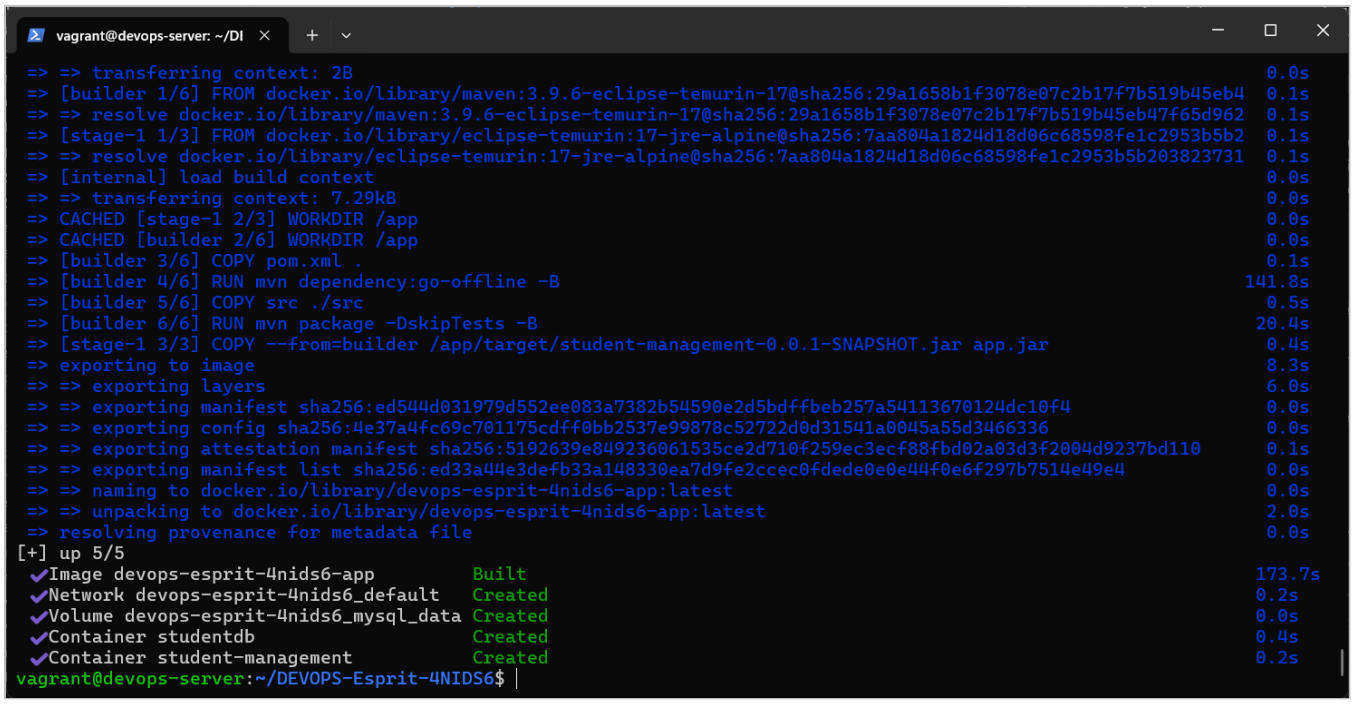
Jenkins plugin installation



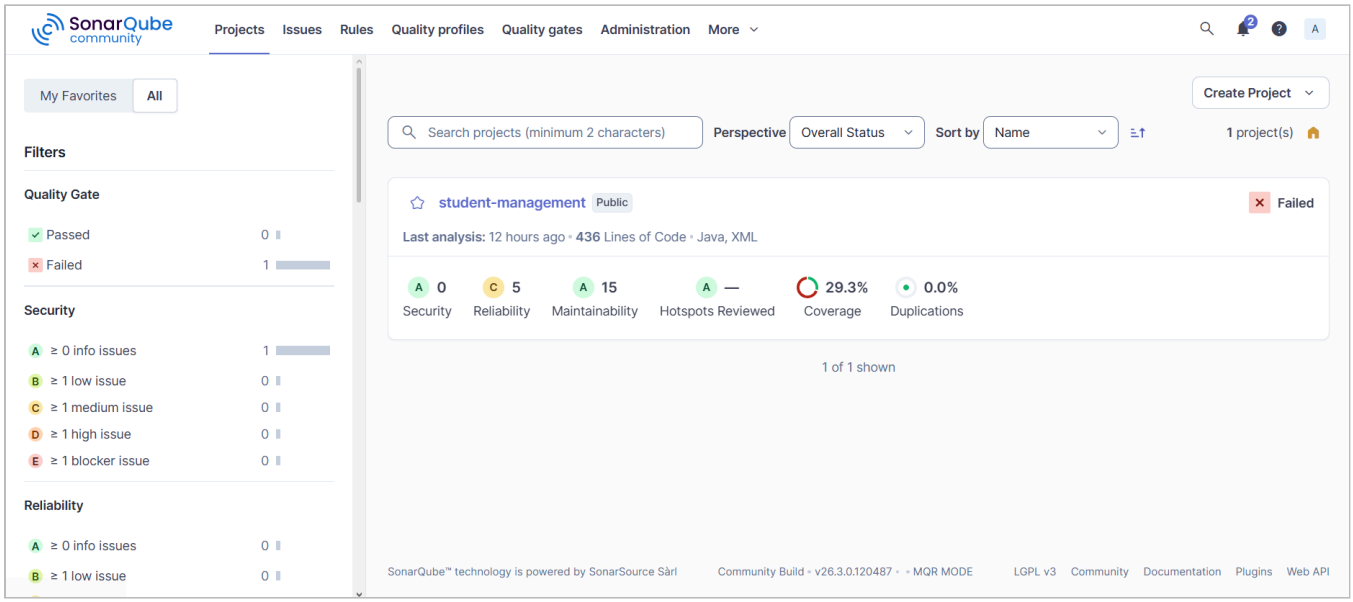
Jenkins dashboard



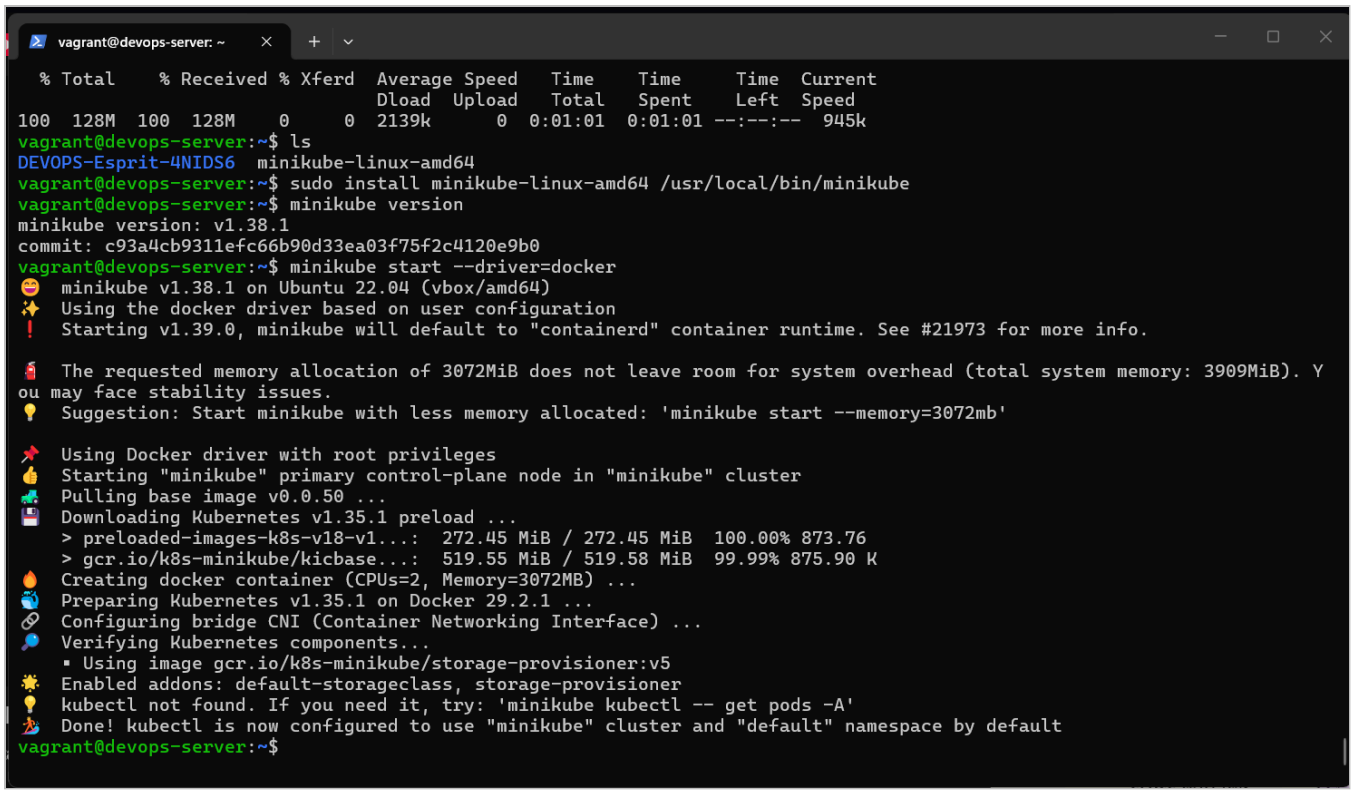
SonarQube interface



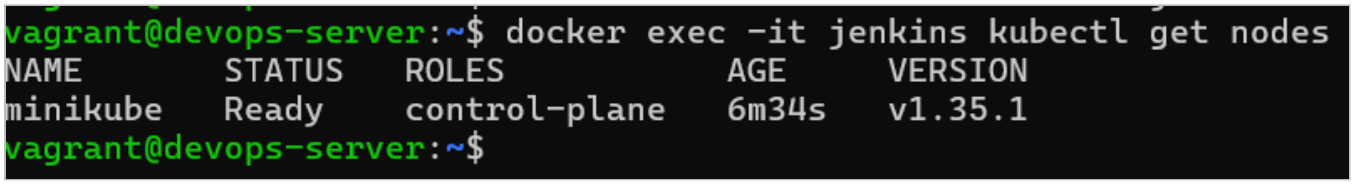
Containers running



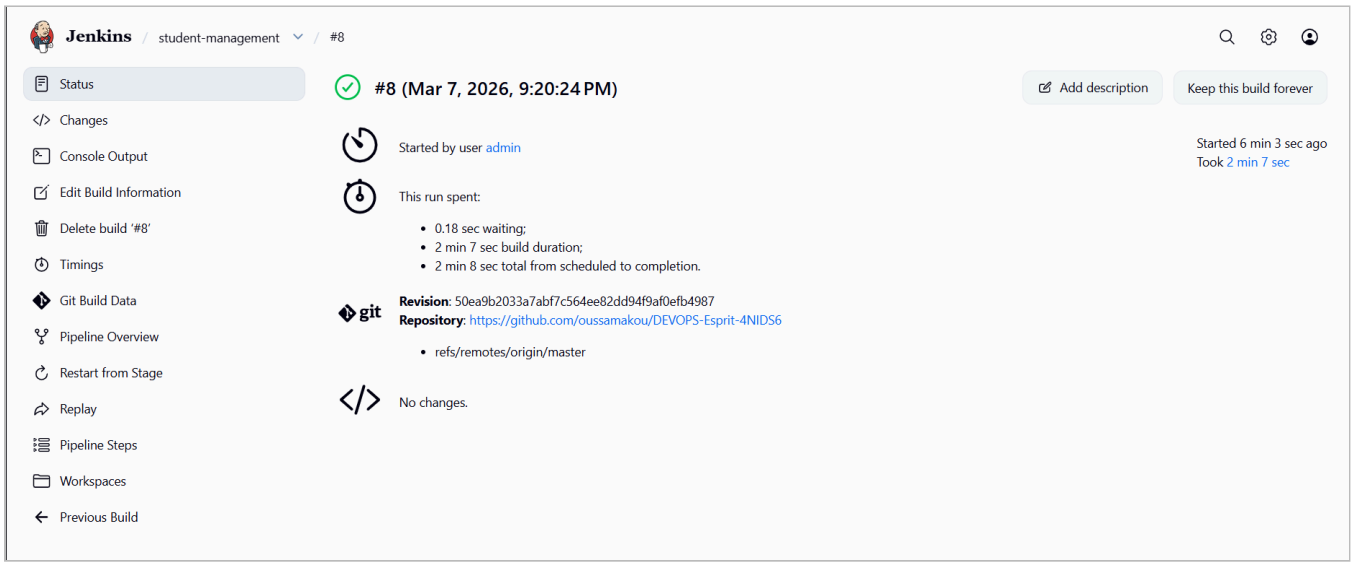
SonarQube 29% coverage



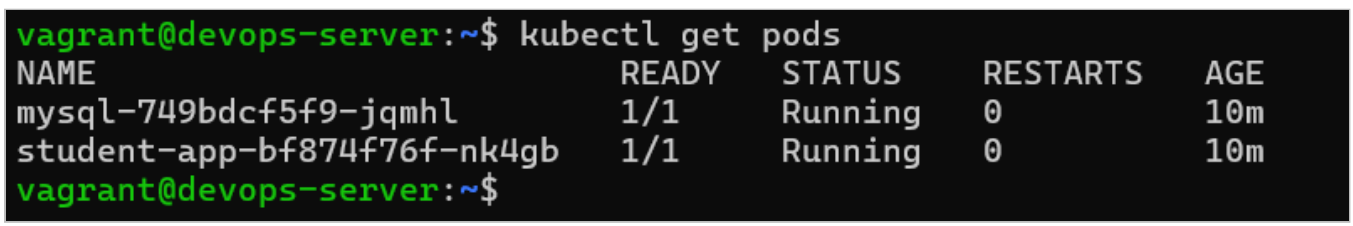
Minikube start



Minikube ready



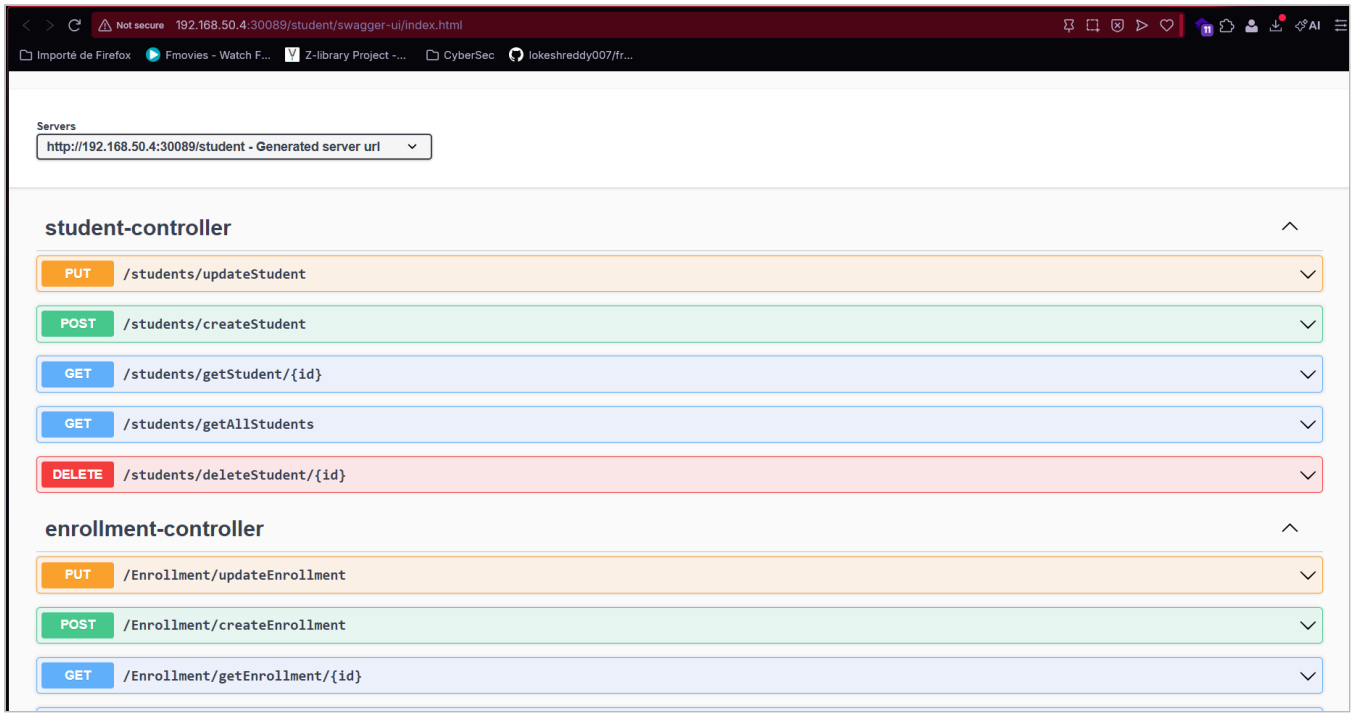
Green Jenkins pipeline



kubectl get pods

```
vagrant@devops-server:~$ kubectl get svc
NAME              TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
kubernetes        ClusterIP   10.96.0.1       <none>        443/TCP          23m
mysql             ClusterIP   10.107.211.25   <none>        3306/TCP         12m
student-service   NodePort    10.97.246.53    <none>        8089:30089/TCP   12m
vagrant@devops-server:~$ minikube service student-service --url
http://192.168.49.2:30089
```

kubectl get svc



Swagger UI